

LINEAMIENTOS PARA EL MAPA CONCEPTUAL COMPLETO DE REACT

Guía formal para estudiantes - preparación previa al desarrollo Frontend de ParkFlow 360

PROPÓSITO DEL TRABAJO

Construir un mapa conceptual que obligue al estudiante a comprender el ecosistema React desde sus fundamentos hasta herramientas modernas de desarrollo profesional. Al finalizar, el estudiante debe poder seguir el desarrollo frontend de ParkFlow 360 realizado por el profesor y transferir el mismo enfoque a su proyecto asignado.

Dato	Definición
Producto a entregar	Mapa conceptual técnico, completo, jerárquico y conectado.
Modalidad	Individual o por grupo, según defina el docente.
Orientación	Aprendizaje previo al desarrollo frontend real.
Tecnología base	React + TypeScript + Vite, consumiendo APIs REST de Spring Boot.
Regla principal	No se evalúa decoración; se evalúa comprensión, jerarquía, relaciones y capacidad de explicar el mapa.

1. Objetivo académico del mapa conceptual

El mapa debe funcionar como una representación visual del conocimiento que el estudiante necesitará para trabajar con React en un proyecto empresarial. No debe ser una colección de definiciones aisladas. Cada concepto debe estar conectado mediante relaciones que expliquen cómo interactúa con los demás.

- Comprender qué problema resuelve React y qué significa construir interfaces basadas en componentes.
- Diferenciar HTML tradicional, JavaScript del navegador, JSX/TSX y componentes React.
- Comprender el flujo de datos, estado, eventos, renderizado y ciclo de actualización de una interfaz.
- Entender cómo React consume un backend REST construido con Spring Boot.

- Conocer las herramientas que permiten escalar desde una pantalla simple a un frontend modular, mantenible, probado y preparado para producción.
- Poder justificar decisiones técnicas durante una defensa individual.

RESULTADO ESPERADO

Después de realizar y defender este mapa, el estudiante debe poder abrir un proyecto React real y reconocer por qué existen componentes, páginas, hooks, rutas, servicios de API, tipos TypeScript, estado, validaciones, consultas al backend, pruebas y herramientas de desarrollo.

2. Regla de construcción: el mapa debe mostrar relaciones, no sólo cajas

Cada nodo debe tener conectores con verbos o frases cortas. No se aceptarán conexiones sin significado como simples líneas. Ejemplos de conectores válidos:

Concepto origen	Conector	Concepto destino
React	construye la UI mediante	Componentes
Componente	recibe datos por	Props
Componente	recuerda información con	State
useState	provoca al actualizarse	Re-render
React Router	asocia URL con	Página/Route
Frontend	consume	API REST Spring Boot
TanStack Query	gestiona	Server State
TypeScript	reduce errores mediante	Tipos estáticos
Vitest + Testing Library	verifican	Comportamiento de componentes

3. Estructura obligatoria del mapa conceptual

El nodo central debe ser REACT. Desde él deben salir, como mínimo, los siguientes doce grandes bloques. Cada bloque debe contener los subtemas indicados en esta guía.

Bloque	Rama principal
A	Fundamentos web previos
B	React y pensamiento por componentes
C	JSX/TSX y renderizado
D	Props, State y flujo de datos
E	Eventos, formularios y validación

F	Hooks y reutilización de lógica
G	Routing y navegación
H	Integración con backend REST
I	Gestión de estado y server state
J	Arquitectura y organización del frontend
K	Testing, debugging y calidad
L	Performance, seguridad y producción

4. Bloque A - Fundamentos web que deben dominar antes de React

HTML y DOM: estructura de la página; etiquetas semánticas; atributos; formularios; relación DOM-navegador.

CSS: selectores, box model, Flexbox, Grid, responsive design y separación entre estructura y presentación.

JavaScript moderno: let/const, funciones, arrow functions, objetos, arrays, destructuring, spread/rest, módulos import/export, promises, async/await.

Navegador: DOM, eventos, HTTP, JSON, localStorage/sessionStorage a nivel conceptual.

Node.js y npm: explicar que React se desarrolla con tooling de Node; package.json; scripts; dependencias y devDependencies.

Vite: servidor de desarrollo y build tool del proyecto React moderno; creación del proyecto y hot reload.

5. Bloque B - React y pensamiento por componentes

React: biblioteca para construir interfaces declarativas basadas en componentes.

Componente: unidad reutilizable de UI y comportamiento. Debe distinguirse página, layout, componente y componente reutilizable.

Composición: un componente puede contener otros componentes; preferir composición sobre componentes gigantes.

Árbol de componentes: Parent/Child; jerarquía visual y jerarquía lógica.

UI declarativa: describir qué debe verse para un estado dado en lugar de manipular manualmente el DOM.

Responsabilidad: cada componente debe tener una razón clara para cambiar y un propósito comprensible.

6. Bloque C - JSX/TSX y renderizado

JSX/TSX: sintaxis que permite describir UI mezclando markup con expresiones JavaScript/TypeScript.

Expresiones: uso de llaves { } para incorporar valores, llamadas y expresiones.

Renderizado condicional: if, operador ternario, &&; mostrar/ocultar partes de la UI.

Listas: map() para renderizar colecciones.

key: identidad estable de elementos al renderizar listas; evitar usar índices sin justificación.

Fragment: agrupar elementos sin añadir nodos innecesarios al DOM.

7. Bloque D - Props, State y flujo de datos

Props: datos que el padre entrega al hijo; son entradas del componente.

State: datos internos que cambian durante la vida de la interfaz.

useState: Hook básico para declarar estado local.

Re-render: cambio de estado puede producir un nuevo render.

Flujo unidireccional: los datos fluyen de padre a hijo; eventos pueden comunicar acciones hacia arriba mediante callbacks.

Lifting state up: mover estado al ancestro común cuando varios componentes deben compartirlo.

Estado derivado: evitar guardar lo que puede calcularse a partir de props/estado existentes.

8. Bloque E - Eventos, formularios y validación

Eventos: onClick, onChange, onSubmit y manejo de eventos en React.

Formularios controlados: inputs cuyo valor se sincroniza con estado React.

Validación cliente: requeridos, formatos y mensajes; distinguir validación UX de validación definitiva del backend.

React Hook Form: herramienta recomendada cuando los formularios crecen y se desea reducir boilerplate.

Zod u otro schema validator: validación tipada y reutilizable; conectar con formularios cuando corresponda.

UX de formularios: disabled, loading, errores por campo, errores globales y confirmación de éxito.

9. Bloque F - Hooks y reutilización de lógica

Reglas de Hooks: llamarlos en el nivel superior de componentes/hooks; no dentro de condicionales arbitrarios.

useState: estado local.

useEffect: sincronización con sistemas externos; no usarlo como solución universal.

useRef: referencias al DOM o valores persistentes que no deben provocar render.

useContext: compartir datos a través del árbol sin prop drilling excesivo.

useReducer: estado complejo y transiciones explícitas.

useMemo / useCallback: optimizaciones; explicar que no deben aplicarse mecánicamente.

Custom Hooks: extraer y reutilizar lógica con nombres que comienzan por use.

10. Bloque G - Routing y navegación

SPA: aplicación que navega entre vistas sin recargar todo el documento.

React Router: biblioteca de routing; relacionar URL con páginas y layouts.

Routes: ruta, path, elemento/página, rutas anidadas.

Parámetros: /clientes/:id y lectura de parámetros.

Navegación: Link/NavLink/useNavigate.

Protección de rutas: concepto de rutas privadas/roles; la autorización real también debe protegerse en backend.

Loading/Error routes: estados de carga y errores asociados a navegación/datos cuando el enfoque de router lo permita.

11. Bloque H - Integración con backend REST

API REST: frontend como cliente del backend Spring Boot.

HTTP: GET, POST, PUT/PATCH, DELETE; status 2xx/4xx/5xx.

JSON: contrato de intercambio de datos.

DTO frontend: tipos/interfaces TypeScript que representan request y response.

fetch / Axios: cliente HTTP; centralizar baseUrl, headers y manejo de errores.

Service/API layer: evitar fetch directo disperso en todos los componentes.

CORS: por qué frontend y backend en orígenes diferentes requieren configuración.

Errores: mostrar correctamente 400, 404, 409, 500 y mensajes del backend.

Loading / Empty / Error / Success: todo consumo remoto debe contemplar estos estados.

12. Bloque I - Gestión de estado y Server State

Local state: estado que sólo interesa a un componente o subtree.

Context: estado compartido simple como tema, sesión o configuración.

Client state: estado puramente de interfaz: filtros locales, modal abierto, preferencias temporales.

Server state: datos que viven en el backend y pueden cambiar fuera del navegador.

TanStack Query: fetching, cache, invalidación, refetch, mutations, stale data y sincronización de server state.

Zustand / Redux Toolkit: alternativas para estado global cliente cuando el problema realmente lo requiere.

Regla de decisión: no llevar todo a estado global; elegir la herramienta según el tipo de estado.

13. Bloque J - Arquitectura y organización profesional del frontend

TypeScript: tipos, interfaces/type aliases, generics básicos, tipos de props y DTOs.

Feature-based structure: organizar por funcionalidad/módulo de negocio, no sólo por tipo de archivo.

Pages: pantallas asociadas a rutas.

Components: piezas reutilizables de UI.

Hooks: lógica reutilizable.

Services/API: acceso al backend.

Types: modelos/DTOs TypeScript.

Shared: componentes/utilidades comunes.

Separación de responsabilidades: la página coordina; el componente presenta; el hook abstrae lógica; el service habla con API.

ParkFlow: módulos sugeridos: auth, clientes, vehículos, estacionamientos, reservas, estancias, pagos, incidencias.

14. Bloque K - Testing, debugging y calidad

React DevTools: inspeccionar árbol, props, estado y renders.

Browser DevTools: Console, Network, Application, Performance.

Vitest: runner de pruebas rápido e integrado con Vite.

React Testing Library: probar comportamiento desde la perspectiva del usuario.

Unit/Component tests: probar funciones, hooks y componentes.

Integration tests: probar interacción entre componentes y servicios simulados.

Mocks: simular APIs/dependencias cuando corresponde.

ESLint/Prettier: consistencia, errores estáticos y formato.

TypeScript strict: usar el compilador como herramienta de calidad, no como obstáculo.

15. Bloque L - Performance, accesibilidad, seguridad y producción

Lazy loading: carga diferida de módulos/rutas cuando aporta valor.

Suspense: límites de espera cuando la herramienta o estrategia lo permita.

Memoización: React.memo/useMemo/useCallback sólo después de identificar un problema.

Code splitting: reducir carga inicial separando bundles.

Accesibilidad: HTML semántico, labels, navegación por teclado, foco, contraste y ARIA cuando sea necesario.

Seguridad: no confiar en el frontend para autorización; evitar exponer secretos; sanitización y prevención de XSS mediante buenas prácticas.

Variables de entorno: configuración con import.meta.env en Vite; nunca guardar secretos del servidor en el frontend.

Build: npm run build; bundle de producción; variables por entorno.

CI/CD: lint, test, build y despliegue automatizado a nivel conceptual.

16. Cómo debe conectarse el mapa con ParkFlow 360

El mapa conceptual no debe quedarse en React genérico. Debe incluir una rama final denominada “Aplicación a ParkFlow 360” que muestre cómo los conceptos anteriores se materializarían en el frontend que desarrollará el profesor.

Concepto React	Ejemplo ParkFlow que debe aparecer
Componentes	ClienteForm, ClienteTable, VehiculoCard, ReservaStatusBadge, AppSidebar.
Props	ClienteRow recibe un ClienteResponse; ReservaStatusBadge recibe estado.
State	filtro de búsqueda, modal abierto, selección de estacionamiento.
Router	/clientes, /clientes/:id, /vehiculos, /reservas, /estancias.
API service	clienteApi.listar(), clienteApi.crear(), vehiculoApi.listarPorCliente(id).
TanStack Query	useQuery para listado; useMutation para crear; invalidación después de guardar.
Formulario	alta/edición de cliente y vehículo con validación.
TypeScript	ClienteResponse, CrearClienteRequest, VehiculoResponse.
Estados remotos	loading, error, empty, success.
Testing	crear cliente, validar error, renderizar listado, navegar a detalle.

17. Cadenas conceptuales obligatorias

Estas cadenas deben poder seguirse visualmente dentro del mapa. Si no existen, el mapa está incompleto.

- React -> Componentes -> Props/State -> Render -> UI
- Evento de usuario -> Handler -> cambio de State -> Re-render
- Formulario -> Validación -> Request DTO -> API REST -> Spring Boot -> Response DTO -> UI
- URL -> React Router -> Page -> Feature/Components
- Page/Hook -> API Service -> HTTP -> Backend
- Server State -> TanStack Query -> Cache -> Query/Mutation -> invalidación/refetch
- TypeScript -> tipos de props/DTOs -> autocompletado + errores en compilación
- Vitest/Testing Library -> prueba -> comportamiento observable -> calidad
- Build Vite -> assets optimizados -> despliegue frontend

18. Formato obligatorio de entrega

Criterio	Requisito mínimo
Nodo central	REACT claramente destacado.
Ramas principales	12 ramas A-L + “Aplicación a ParkFlow 360”.
Subconceptos	Todos los conceptos obligatorios señalados en esta guía.
Conectores	Toda relación debe llevar un verbo/frase que explique el vínculo.
Jerarquía	Fundamentos -> construcción UI -> datos -> arquitectura -> calidad -> producción.
Ejemplos	Al menos 10 ejemplos relacionados con ParkFlow o el proyecto asignado.
Claridad	Debe poder leerse en impresión o PDF a tamaño normal.
Defensa	Cualquier integrante debe poder explicar cualquier rama sin leer definiciones de memoria.
Fuentes	Incluir al final referencias oficiales utilizadas.
Originalidad	No copiar una imagen de Internet ni entregar un mapa generado automáticamente sin revisión y defensa.

19. Procedimiento recomendado para construir el mapa

Paso	Acción
1	Leer primero la guía completa y marcar conceptos desconocidos.
2	Crear el nodo central REACT.
3	Dibujar sólo las 12 ramas principales y validar que la jerarquía tenga sentido.
4	Completar fundamentos (A-D) antes de pasar a herramientas avanzadas.
5	Agregar Hooks, routing, backend y estado remoto (E-I).
6	Agregar arquitectura, testing y producción (J-L).
7	Conectar conceptos entre ramas; por ejemplo, Router con Pages, TanStack Query con API Service y TypeScript con DTOs.
8	Agregar la rama ParkFlow 360 con ejemplos concretos.
9	Realizar una revisión técnica: eliminar conceptos repetidos y líneas sin significado.
10	Ensayar defensa: explicar el recorrido desde un clic del

	usuario hasta Spring Boot y de vuelta a la pantalla.
--	--

20. Rúbrica sugerida de evaluación

Criterio	Peso	Qué se evalúa
Cobertura técnica	25%	Incluye fundamentos, React, Hooks, Router, API, estado, arquitectura, testing y producción.
Jerarquía conceptual	20%	Conceptos organizados de general a específico y con niveles coherentes.
Relaciones y conectores	20%	Las líneas expresan relaciones técnicas reales; no son decorativas.
Aplicación a ParkFlow/proyecto	15%	Puede transferir los conceptos a pantallas, rutas, DTOs y casos reales.
Precisión técnica	10%	No confunde props con state, client state con server state, React con Vite, etc.
Defensa individual	10%	Explica decisiones y recorridos sin depender de lectura literal.

21. Preguntas que el estudiante debe poder responder al defender el mapa

- ¿Qué problema resuelve React frente a manipular manualmente el DOM?
- ¿Cuál es la diferencia entre un componente, una página y un layout?
- ¿Cuál es la diferencia entre props y state?
- ¿Qué provoca un re-render y qué significa realmente?
- ¿Cuándo usarías useEffect y cuándo NO?
- ¿Por qué una lista necesita key estable?
- ¿Qué diferencia existe entre client state y server state?
- ¿Qué problema resuelve TanStack Query?
- ¿Por qué no conviene hacer fetch directamente en veinte componentes diferentes?
- ¿Cómo viaja un CrearClienteRequest desde React hasta Spring Boot?
- ¿Qué diferencia existe entre la validación del formulario y las reglas de negocio del backend?
- ¿Qué papel cumple React Router?
- ¿Qué aporta TypeScript al frontend?
- ¿Por qué una API puede devolver 409 y cómo debe reaccionar la interfaz?
- ¿Qué pruebas escribirías para el formulario de cliente?
- ¿Qué datos jamás deberían tratarse como “secreto” dentro del bundle frontend?
- ¿Qué parte del frontend organizarías por feature en ParkFlow?
- ¿Qué herramientas usarías para diagnosticar una llamada HTTP que falla?

- ¿Cuándo considerarías Zustand/Redux Toolkit y cuándo serían innecesarios?
- Explica el flujo completo: clic en “Guardar Cliente” -> backend -> respuesta -> actualización del listado.

22. Errores que harán que el mapa se considere incompleto

- Presentar React como un lenguaje de programación.
- Confundir React con Vite, Node.js o npm.
- Colocar todos los conceptos al mismo nivel sin jerarquía.
- No diferenciar props, state, context y server state.
- Mostrar useEffect como herramienta para “todo”.
- No explicar cómo se consume el backend Spring Boot.
- Ignorar estados loading/error/empty/success.
- Usar Redux/Zustand como solución obligatoria para cualquier estado.
- No incluir testing, accesibilidad, seguridad y build.
- Entregar únicamente definiciones sin conectores ni ejemplos.

23. Checklist final antes de entregar

- ☐ ¿El nodo central es REACT?
- ☐ ¿Aparecen las 12 ramas principales?
- ☐ ¿Cada línea tiene un conector con significado?
- ☐ ¿Incluye JSX/TSX, componentes, props, state y eventos?
- ☐ ¿Incluye Hooks y reglas de uso?
- ☐ ¿Incluye React Router?
- ☐ ¿Incluye consumo de API REST y DTOs?
- ☐ ¿Diferencia client state de server state?
- ☐ ¿Incluye TanStack Query?
- ☐ ¿Incluye TypeScript?
- ☐ ¿Incluye estructura por features?
- ☐ ¿Incluye testing y debugging?
- ☐ ¿Incluye performance, accesibilidad, seguridad y build?
- ☐ ¿Incluye ejemplos de ParkFlow?
- ☐ ¿Todo integrante puede explicar el recorrido end-to-end?

24. Referencias oficiales recomendadas

- React Learn: <https://react.dev/learn>
- React Hooks Reference: <https://react.dev/reference/react/hooks>
- Vite Guide: <https://vite.dev/guide/>
- React Router: <https://reactrouter.com/>
- TanStack Query: <https://tanstack.com/query/latest>
- Vitest: <https://vitest.dev/>

- Testing Library: <https://testing-library.com/>
- TypeScript: <https://www.typescriptlang.org/docs/>

MENSAJE FINAL AL ESTUDIANTE

Este mapa conceptual no es un trabajo decorativo. Debe convertirse en tu “mapa mental” del frontend. Cuando el profesor empiece ParkFlow 360, deberás poder reconocer cada concepto dentro del código y replicar la misma ingeniería en tu propio proyecto.